

Backend Architecture & API

Source Document

This document describes the backend architecture of the Multi-Factor Stock Selection System, a Flask-based quantitative analysis platform. It covers the three-layer architecture, API endpoints, service layer design, data models, and the factor optimization workflow.

1. Application Factory Pattern

The Flask application uses the factory pattern. `create_app()` in `app/__init__.py` registers all blueprints, initializes extensions (SQLAlchemy, Redis, SocketIO, CORS), and configures the application from `config.py`. The pattern supports multiple environments (development, production, testing) through configuration classes.

```
def create_app(config_name='default'):
    app = Flask(__name__)
    app.config.from_object(config[config_name])

    db.init_app(app)
    socketio.init_app(app, message_queue=app.config['SOCKETIO_MESSAGE_QUEUE'])

    from app.api.ml_factor_api import ml_factor_bp
    app.register_blueprint(ml_factor_bp)  # /api/ml-factor/*

    from app.routes.ml_factor_routes import ml_factor_routes
    app.register_blueprint(ml_factor_routes)  # /ml-factor/*

    return app
```

2. Three-Layer Architecture

The system follows a classic three-layer architecture: API layer handles HTTP requests and parameter validation; Service layer contains core business logic (factor computation, model training, scoring, optimization); Data layer provides SQLAlchemy ORM models mapped to MySQL tables. Services use lazy-initialized singleton instances for efficiency.

```
# API Layer (ml_factor_api.py)
@ml_factor_bp.route('/factors/auto-weight', methods=['POST'])
def get_auto_weights():
    data = request.get_json()
    result = get_factor_optimizer().get_optimized_weights(
        evaluation_date=data.get('evaluation_date'),
        start_date=data.get('start_date'),
        forward_period=data.get('forward_period', 5),
        weight_method=data.get('weight_method', 'ic_ir_weighted'),
        ic_ir_threshold=data.get('ic_ir_threshold', 0.3),
        max_correlation=data.get('max_correlation', 0.7),
        min_factors=data.get('min_factors', 3),
        max_factors=data.get('max_factors', 10)
    )
    return jsonify(result)

# Service Layer (factor_optimizer.py)
class FactorOptimizer:
    def compute_rank_ic(self, factor_id, trade_date, forward_period=5): ...
    def compute_rolling_ic_statistics(self, factor_id, start, end, period): ...
    def analyze_all_factors(self, start, end, periods): ...
    def select_factors(self, data, ic_ir_threshold, max_correlation, min, max): ...
    def get_optimized_weights(self, evaluation_date, start_date, forward_period): ...
    def get_effectiveness_report(self, factor_id, forward_period, limit): ...
```

3. Factor Optimization API Endpoints

Three new API endpoints were added to support automatic factor optimization. All endpoints use JSON for request/response and follow consistent error handling patterns.

```

# Run full optimization (IC analysis + selection + weights)
POST /api/ml-factor/factors/optimize
{
  "start_date": "2025-01-01", "end_date": "2025-04-01",
  "forward_periods": [1, 5, 20],
  "ic_ir_threshold": 0.3, "max_correlation": 0.7,
  "min_factors": 3, "max_factors": 10,
  "weight_method": "ic_ir_weighted"
}

# Query persisted effectiveness data
GET /api/ml-factor/factors/effectiveness?factor_id=momentum_5d&forward_period=5&limit=100

# Get auto-computed weights for a given date
POST /api/ml-factor/factors/auto-weight
{
  "evaluation_date": "2025-04-01", "start_date": "2025-01-01",
  "forward_period": 5, "weight_method": "ic_ir_weighted",
  "ic_ir_threshold": 0.1, "max_correlation": 0.7,
  "min_factors": 2, "max_factors": 8
}

# Response:
{ "success": true,
  "weights": {"volatility_20d": 0.423, "momentum_5d": 0.301, ...},
  "selected_factors": {...}, "evaluation_date": "2025-04-01" }

```

4. Stock Scoring & Backtest Integration

The scoring engine supports four methods: `equal_weight`, `factor_weight`, `rank_ic`, and `ml_ensemble`. `rank_ic` and `ml_ensemble` were previously TODO stubs and have been fully implemented. The backtest engine now accepts `scoring_method` in `strategy_config`, enabling automatic weight optimization within backtest runs. Each rebalance date triggers `_rank_ic_scoring` which fetches optimized weights from `FactorOptimizer` based on historical IC data.

```

# Backtest strategy_config with auto-weight
POST /api/ml-factor/backtest/run
{
  "strategy_config": {
    "selection_method": "factor_based",
    "scoring_method": "rank_ic",
    "factor_list": ["momentum_1d", "momentum_5d", "momentum_20d",
                  "volatility_20d", "volume_ratio_20d", "price_to_ma20"],
    "top_n": 20, "transaction_cost": 0.001
  },
  "start_date": "2025-01-03", "end_date": "2026-02-12",
  "initial_capital": 1000000, "rebalance_frequency": "monthly"
}

```

5. Data Model

Key database tables include: `factor_effectiveness` (stores IC statistics with composite primary key `factor_id` + `evaluation_date` + `forward_period`), `factor_values` (computed factor values per stock per date), `ml_model_definition` (ML model metadata with `feature_importance` JSON column), and `stock_daily_history` (1.47M rows of daily OHLCV data covering 2025-2026).

```
# factor_effectiveness table (new) - stores IC analysis results
class FactorEffectiveness(db.Model):
    __tablename__ = 'factor_effectiveness'
    factor_id = Column(String(50), primary_key=True)
    evaluation_date = Column(Date, primary_key=True)
    forward_period = Column(Integer, primary_key=True)
    ic_mean = Column(DECIMAL(10, 6))
    ic_std = Column(DECIMAL(10, 6))
    ic_ir = Column(DECIMAL(10, 6))          # IC_IR = ic_mean / ic_std
    ic_win_rate = Column(DECIMAL(5, 2))
    latest_ic = Column(DECIMAL(10, 6))
    sample_count = Column(Integer)
    factor_direction = Column(String(10))   # positive / negative
    extra_info = Column(JSON)              # IC series data
```

6. Frontend Integration

The factor optimization page (/ml-factor/optimization) provides a full workflow: configure parameters -> run auto-weight analysis -> view IC statistics and weight charts -> run backtest comparison. The backtest page (/ml-factor/backtest) now includes a scoring method dropdown (rank_ic/equal_weight/manual) with a "Calculate Optimal Weights" button that auto-selects factors from the auto-weight API result.

```
// Frontend: Calculate optimal weights then run backtest
async function calcAutoWeights() {
    const resp = await axios.post('/api/ml-factor/factors/auto-weight', {
        evaluation_date: endDate, start_date: startDate,
        forward_period: 5, ic_ir_threshold: 0.1,
        max_correlation: 0.7, min_factors: 2, max_factors: 8
    });
    cachedWeights = resp.data.weights;
    // Auto-check corresponding factor checkboxes
    const checkboxes = document.querySelectorAll('#factorSelection input[type="checkbox"]');
    checkboxes.forEach(cb => { cb.checked = Object.keys(cachedWeights).includes(cb.value); });
}

// Backtest runs with scoring_method='rank_ic', backend auto-fetches weights
const strategyConfig = {
    selection_method: 'factor_based',
    scoring_method: 'rank_ic', // auto-weight enabled
    factor_list: config.selected_factors,
    ...
};
```